Carbon Debt Computing

A Deadline-Aware Carbon Debt Controller

Project Proposal and Architecture Document

Challenge 9: Carbon Debt Computing (individual challenge)

Field	Details
Student name	[STUDENT NAME]
Registration number	[REGISTRATION_NO]
Programme	[PROGRAMME]
Course	[COURSE CODE / NAME]
Faculty	[FACULTY NAME]
Team	Individual submission
Date	28 October 2026

Contents

1. Problem statement and objectives 3
Objectives 3
2. Proposed novelty 4
3. System architecture 5
Component breakdown 5
4. Flowchart and scheduling sequence 7
5. Hardware and software requirements 9
Hardware 9
Software, APIs and frameworks 9
6. Algorithms and formulas 10
Policies 10
Progressive throttling 10
7. Measurement methodology 11
8. Experimental design and results 12
Design 12
Policy comparison (mean over seeds) 12
Demonstration scenario 12
Findings 13
9. Limitations and future scope 14
Limitations 14
Future scope 14
10. References 15

Right-click the contents list and choose Update Field if page numbers are missing.

1. Problem statement and objectives

Cloud platforms increasingly shift flexible workloads to times when the electricity grid is cleaner. Emergency workloads, such as a hospital triage-model failover, a flood-alert ensemble recomputation or a fraud-detection retrain, cannot wait. They must run immediately, even when the grid is fossil-heavy. Existing carbon-aware schedulers handle these as exceptions and then forget them: the excess emissions are never recorded against the system, and nothing changes afterwards to compensate.

This project treats those excess emissions as **carbon debt**. Debt is recorded in an auditable ledger, and the scheduler is obliged to repay it by moving future flexible work into green (low-carbon) periods. While the debt is outstanding, non-critical workloads are progressively throttled, and no job is ever pushed past its deadline.

Objectives

1. **O1 — Debt ledger.** Record policy-defined excess emissions of high-carbon execution in an append-only, hash-chained ledger.
2. **O2 — Priority inversion.** While debt is outstanding, direct eligible flexible capacity into strictly green time slots until the balance returns to zero.
3. **O3 — Throttling.** Automatically throttle or defer non-critical workloads when debt crosses configured thresholds, without ever throttling emergencies.
4. **O4 — Verified repayment.** Grant repayment credit only for deliberate, deadline-meeting deferrals against a counterfactual baseline recorded in advance.
5. **O5 — Deadline safety and prediction.** Never defer work past its deadline, and project the debt path over a forecast horizon.
6. **O6 — Measurable outcomes.** Demonstrate live operation and compare against baselines with reproducible quantitative experiments.

2. Proposed novelty

- **Debt with memory.** Unavoidable emergency emissions persist as a debt that changes later scheduling decisions. In typical carbon-aware schedulers, decisions depend only on the current or forecast intensity.
- **Verified counterfactual credits.** The baseline intensity is fixed in a hash-chained DEFERRAL_RECORDED event when the controller defers a job. The credit is max(0, E·I_baseline − actual emissions), granted once, only if the deadline is met and no operator override occurred. Surplus credit is not banked, so green work cannot pre-pay future emergencies.
- **Repayment-value priority inversion.** During repayment, waiting jobs are ranked by the debt they can repay (energy × intensity drop) instead of nominal priority.
- **Deadline-safe progressive throttling.** Debt bands map to 100/50/20%/paused capacity for normal and background work. A level is applied only if the job still finishes two slots before its deadline, and throttling's energy penalty (static power) is modelled honestly.
- **Predictive debt controller.** An earliest-deadline-first capacity reservation over the forecast, a green-only window restriction while in debt, a feasibility repair pass, and a debt projection that triggers proactive throttling.
- **One deterministic engine for live operation and experiments.** The dashboard and the policy comparison are produced by the same Decimal-exact code, so experimental results are exactly what the system does.

3. System architecture

The system is a simulated cloud scheduler that runs locally by default, with an optional real cloud deployment of the controller and dashboard. All logic runs in the backend; the browser only displays persisted data.

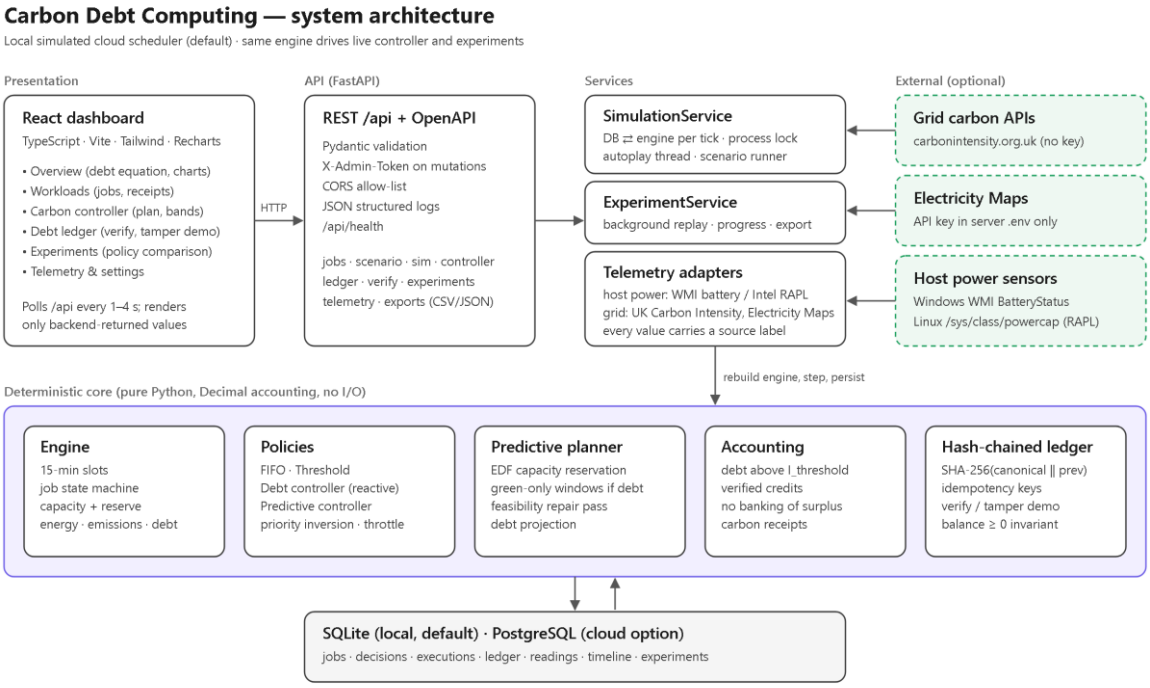


Figure 1. Block diagram of the system.

Component breakdown

Component	Technology	Responsibility
Dashboard	React, TypeScript, Vite, Tailwind, Recharts	Overview, Workloads, Carbon controller, Debt ledger, Experiments, Telemetry & settings
REST API	FastAPI, Pydantic	Validated endpoints, OpenAPI docs, admin token, CORS, structured logs, health check
SimulationService	Python, SQLAlchemy	Per 15-minute tick: rebuild engine from DB, step, persist all new records in one transaction; autoplay; scenario
Engine and policies	Pure Python, Decimal	Job state machine, capacity and emergency reserve, energy/emissions/debt, FIFO/threshold/debt/predictive policies
Predictive planner	Pure Python	EDF reservation, green-only windows under debt, repair pass, debt projection
Accounting and ledger	SHA-256, canonical JSON	Debt and credit rules, idempotency, balance ≥ 0, verification and tamper detection
Experiment runner	Pure Python	Identical seeded traces and curves per policy; metrics; parameter sweeps; CSV/JSON export
Live mode	httpx, psutil, PDH (ctypes)	India Energy Atlas grid feed, Windows process monitor, measured CPU-package energy, bounded CPU workers

Component	Technology	Responsibility
Persistence	SQLite / PostgreSQL	Jobs, decisions, execution attempts, ledger, readings, timeline, experiments

4. Flowchart and scheduling sequence

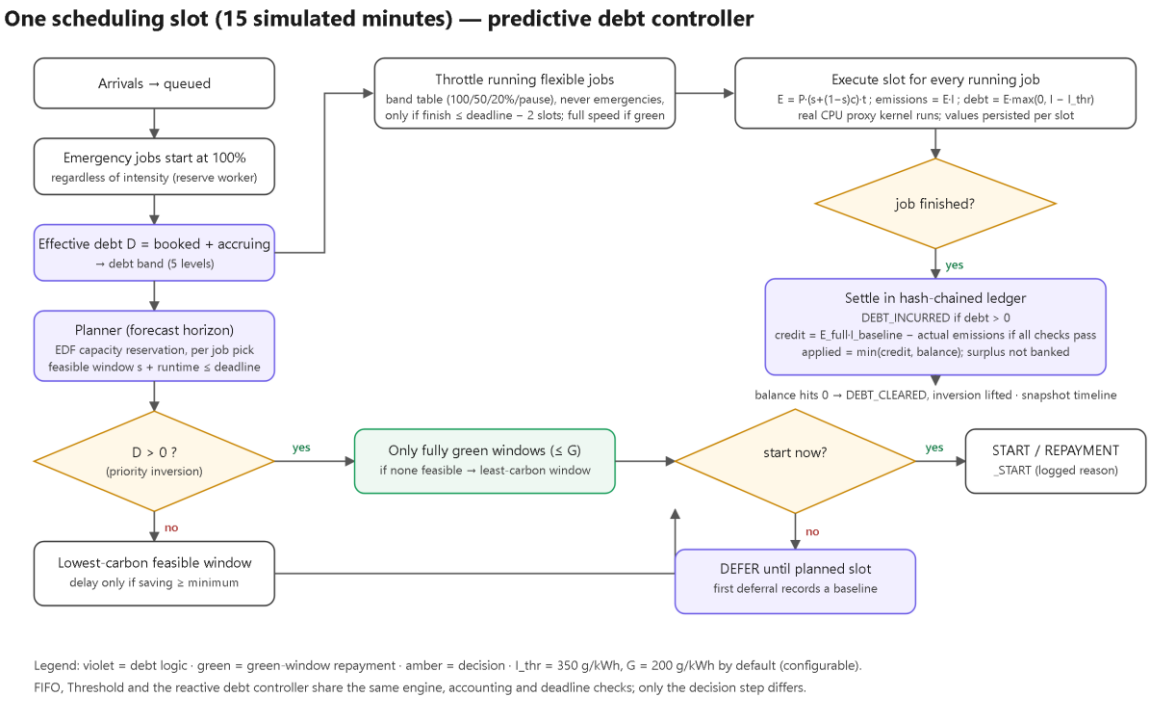


Figure 2. Processing of one scheduling slot by the predictive debt controller.

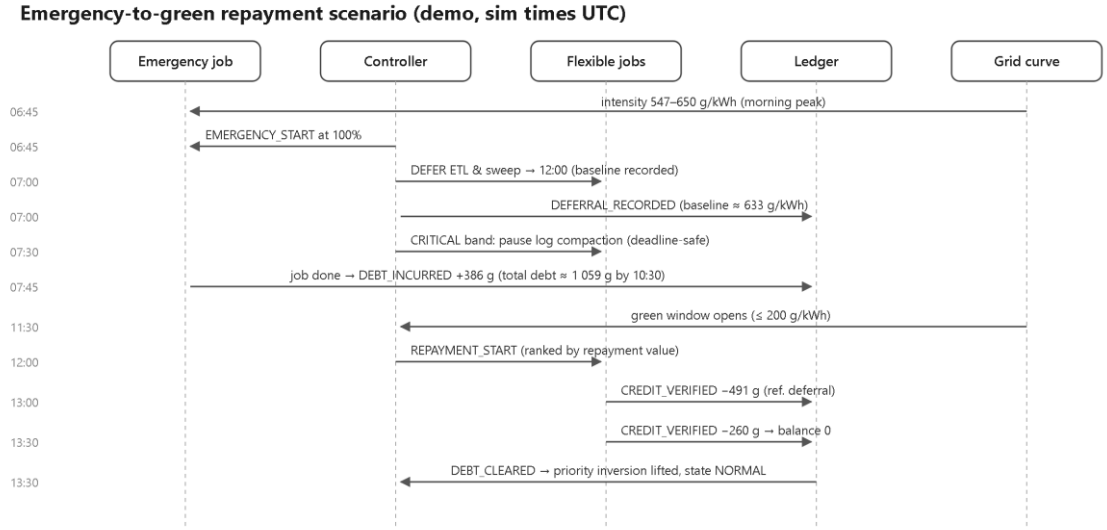


Figure 3. Emergency-to-green repayment scenario (demonstration).

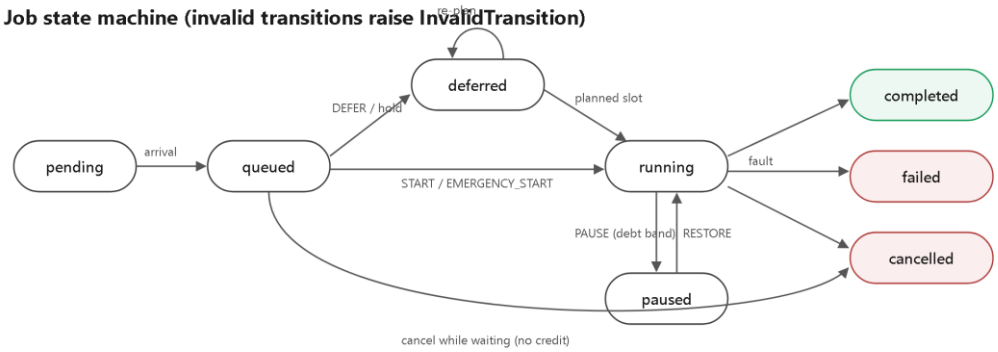


Figure 4. Job state machine; any other transition is rejected.

5. Hardware and software requirements

Hardware

Item	Used for development / demo	Notes
Laptop	Windows 11, Intel Core i5-13500H, battery-powered laptop	Any machine with Python 3.11 and Node 20 works
Power telemetry	Windows Energy Meter counter (Intel RAPL, CPU package)	Measured package energy; no admin rights needed
Optional	Linux host with Intel RAPL	CPU package power; needs read permission
Cloud (optional)	1 small VM (e.g. Azure B2ats_v2 / B1ms)	Docker Compose + Caddy HTTPS

Software, APIs and frameworks

Layer	Software
Backend	Python 3.11, FastAPI 0.115, Pydantic 2.10, SQLAlchemy 2.0, Uvicorn, httpx, psutil, pytest
Frontend	React 18, TypeScript 5.6, Vite 5, Tailwind CSS 3.4, Recharts 2.13, lucide-react
Data	SQLite (default), PostgreSQL via psycopg 3 (cloud option)
Carbon-intensity API	India Energy Atlas (api.energymap.in, X-API-Key; free Sandbox/Student plans, all-India)
Telemetry	psutil (processes, CPU, memory); Windows Energy Meter via PDH; Linux powercap RAPL
Deployment	Docker, Docker Compose, nginx, Caddy (Let's Encrypt TLS), optional Render blueprint

6. Algorithms and formulas

Time advances in 15-minute slots. Units: kWh, gCO_2e/kWh, gCO_2e, W. Accounting uses Decimal arithmetic, quantised per slot.

```
P(c)            = P_declared × (s + (1 − s)·c)            s = 0.30
energy_slot     = P(c) × minutes / 60 / 1000              [kWh]
gross_emissions = Σ energy_slot × I_slot                  [gCO₂e]
debt            = Σ energy_slot × max(0, I_slot − I_thr)  I_thr = 350
est_savings     = E × max(0, I_now − I_candidate)
credit          = max(0, E_full × I_baseline − gross_emissions)
credit_applied  = min(credit, outstanding);  outstanding = Σdebt − Σcredit_applied
≥ 0
hash_n          = SHA-256(canonical_json(record_n fields ‖ prev_hash))
```

Policies

- **Emergency precedence (all policies).** Start immediately at 100%; one worker is reserved; never throttled.
- **Threshold.** Run if I ≤ I_thr, otherwise defer one slot while start + runtime ≤ deadline remains true.
- **Debt controller.** If debt > 0, flexible jobs start only in green slots (I ≤ 200), ranked by repayment value; otherwise the threshold rule applies.
- **Predictive controller.** Every slot, reserve capacity earliest-deadline-first and choose the lowest-carbon feasible window (only fully green windows while in debt, if any fit), with a 2-slot deadline margin and a repair pass. Delay only for a saving of at least 5 g. Project the debt path and raise the throttle band early if debt cannot be repaid within the horizon.

Progressive throttling

Effective debt	State	Normal	Background
0	Normal	100%	100%
0 – 100 g	Repaying	100%	100%
100 – 400 g	Soft throttle	100%	50%
400 – 800 g	Hard throttle	50%	20%
≥ 800 g	Critical	20%	Paused

Throttling applies only outside green windows, never to emergencies, and only when the job still finishes two slots before its deadline.

7. Measurement methodology

- **Labels.** Every value is SIMULATED, MANUAL_OVERRIDE, LIVE_ACTUAL / LIVE_FORECAST, ESTIMATED, MEASURED or UNAVAILABLE. A simulated value is never shown as live.
- **Energy (ESTIMATED).** Declared job power × runtime with a static/dynamic throttle model. A real SHA-256 proxy kernel runs for each job-slot and its CPU time is recorded as execution evidence, not as job energy.
- **Grid intensity.** Simulation mode: a deterministic, seeded simulated curve (solar-midday profile, 3% noise). Live mode: the all-India reading from India Energy Atlas (polled every 20 minutes within the free 100-calls/day quota), with provider, region, timestamp and freshness displayed; the provider models the fuel mix (±3%).
- **Live energy (MEASURED, attributed).** CPU-package energy from the Windows Energy Meter counter (Intel RAPL), shared to each managed job by its share of busy CPU time; processes, CPU and memory from psutil. Ordinary Windows programs are only observed.
- **Uncertainty.** Declared power may differ from reality by tens of percent; provider intensities are averages, not marginal factors; forecast uncertainty is not modelled. Comparisons between policies are fair because they share identical inputs.

8. Experimental design and results

All results in this section come from deterministic simulation (SIMULATED energy and grid). They were produced by the implemented system with tools/generate_results.py and are not hardware measurements.

Design

- Four policies on identical inputs: FIFO/priority, threshold deferral, the debt controller without forecast, and the predictive debt controller.
- 10 seeds. Each trace has 36 jobs (4 emergency, 18 normal, 14 background); 4 workers with 1 reserved; solar-midday grid; I_thr = 350, green ≤ 200 gCO_2e/kWh.
- Metrics: gross emissions, debt incurred, credits, final and peak debt, deadline misses, delay, energy, throttle actions and decision time.

Policy comparison (mean over seeds)

Policy	Emissions (g)	vs FIFO	Debt incurred (g)	Final debt (g)	Misses/run	Delay (min)
FIFO / priority	20,637	—	5,385	5,385	0.2	24
Threshold deferral	17,588	-14.8%	2,965	469	1.3	236
Debt controller (no forecast)	16,880	-18.2%	3,160	72	3.8	422
Predictive debt controller	17,197	-16.7%	2,880	248	1.3	408

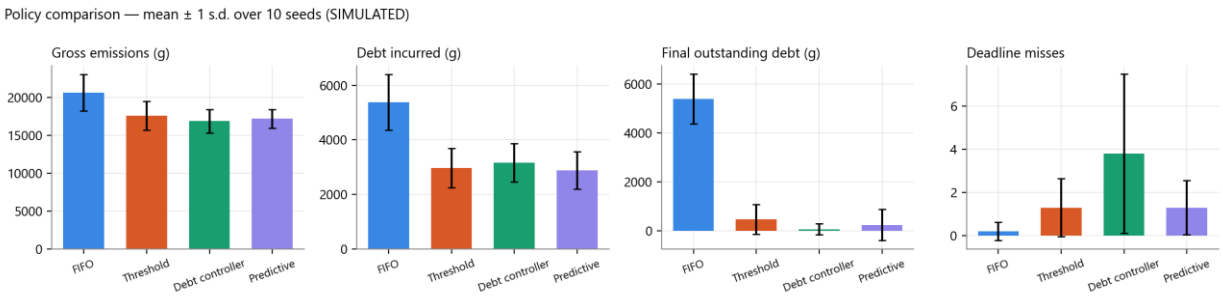


Figure 5. Mean ± 1 standard deviation across seeds (SIMULATED).

Demonstration scenario

Policy	Gross emissions (g)	Debt incurred (g)	Final debt (g)	Deadline misses
FIFO / priority	4,742	1,980	1,980	0
Threshold deferral	3,444	1,047	0	0
Debt controller (no forecast)	3,187	1,064	0	0
Predictive debt controller	3,122	1,059	0	0

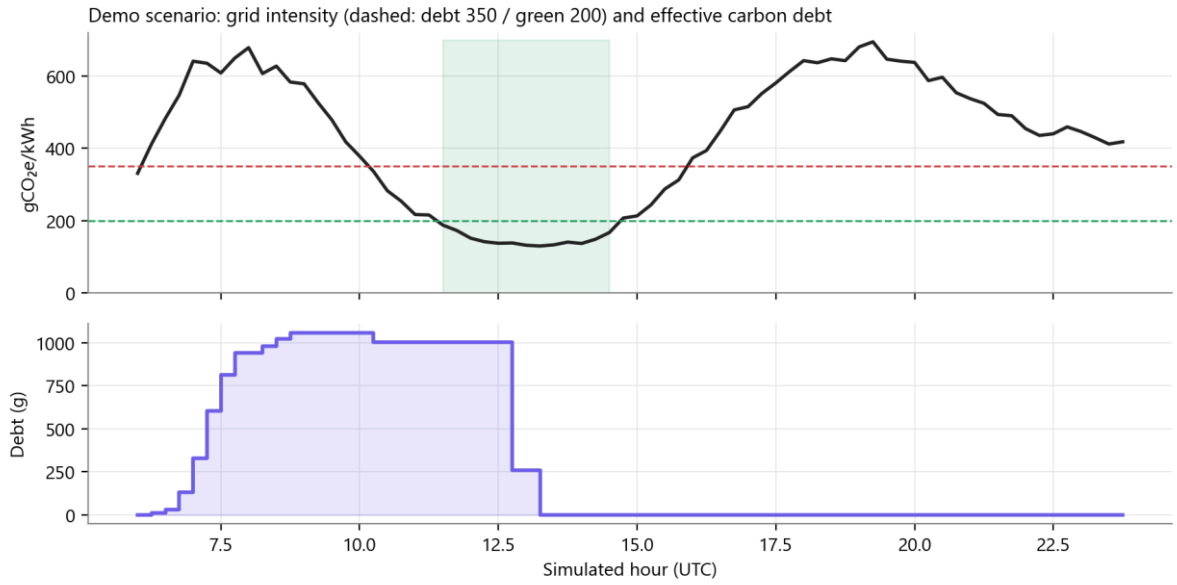


Figure 6. Demo scenario under the predictive controller: debt created in the morning peak and repaid in the green window.

Findings

- Against FIFO, the predictive controller changed gross emissions by -16.7% and debt incurred by -46.5%, with 1.3 deadline misses per run (threshold: 1.3).
- Forecasting matters for deadlines: the debt controller without a forecast missed 3.8 per run, because jobs pile up at their last feasible slot. It reached slightly lower emissions (16,880 g) and final debt (72 g).
- The cost of carbon awareness is delay for flexible work (average completion delay 408 min vs 24 min for FIFO), plus 3.0% more energy from throttling.
- In the demo, 1,059 g of debt was fully repaid by verified credits, with no missed deadlines; the hash chain verified (14 entries).

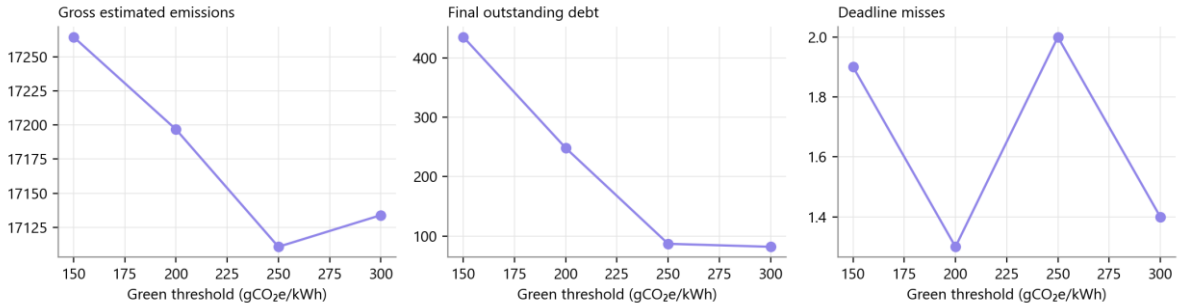


Figure 7. Sweep of the green threshold for the predictive controller.

9. Limitations and future scope

Limitations

- The cluster is simulated locally; workloads are not dispatched to real cloud servers.
- Job energy is estimated from declared power; no meter is attributed to simulated jobs.
- The default grid curve is illustrative, and forecast uncertainty is not modelled.
- Debt is a policy-defined quantity; credits record avoided future emissions and do not erase past emissions.
- The hash chain is tamper-evident only relative to an externally kept head hash.
- Throttling is scheduler-level (capacity share), with a linear power model.

Future scope

- Dispatch decisions to Kubernetes Jobs or a cloud batch queue, using node power from IPMI/Redfish or provider APIs.
- Probabilistic forecasts with a risk-aware planner, and marginal emission factors.
- An OS-level cgroups v2 CPU-quota adapter on Linux hosts.
- Periodic anchoring of the ledger head to an external append-only log.
- Multi-tenant debt budgets per application or team.

10. References

7. India Energy Atlas developer documentation, https://www.energymap.in/developer (carbon-intensity/latest, forecast/carbon-intensity, authentication, quotas).
8. psutil documentation, https://psutil.readthedocs.io/; Microsoft, Performance Data Helper (PDH) API and the Energy Meter counter set.
9. Linux kernel documentation, Power Capping Framework (powercap / Intel RAPL), https://docs.kernel.org/power/powercap/powercap.html.
10. FastAPI documentation, https://fastapi.tiangolo.com/; SQLAlchemy 2.0 documentation, https://docs.sqlalchemy.org/.
11. React, https://react.dev/; Recharts, https://recharts.org/; Vite, https://vitejs.dev/.
12. Caddy web server (automatic HTTPS), https://caddyserver.com/docs/; Microsoft Azure for Students, https://azure.microsoft.com/free/students/.